

Funkce: definice, volání a rozsah platnosti

Základy programování

Matouš Cejnek

U12110, FS, ČVUT v Praze

Co si dnes odnesete?

- Definovat a zavolat vlastní funkci.
- Rozlišit parametr, argument a návratovou hodnotu.
- Předpovědět pořadí provádění programu s funkcemi.
- Vysvětlit lokální a globální rozsah platnosti.
- Rozdělit delší program na malé spolupracující části.

Na co navazujeme?

- Proměnné uchovávají hodnoty různých datových typů.
- Podmínky rozhodují a cykly opakují příkazy.
- Teď budeme celé kroky algoritmu **pojmenovávat a skládat**.

Motivace

Dobrá funkce převádí jasně určené vstupy na výstup a skrývá podrobnosti výpočtu.

Když stejný výpočet píšeme opakovaně

```
area_1 = 3.14159 * (d_1 / 2) ** 2
```

```
area_2 = 3.14159 * (d_2 / 2) ** 2
```

```
area_3 = 3.14159 * (d_3 / 2) ** 2
```

- Oprava vzorce se musí provést na více místech.
- Záměr kódu není pojmenovaný.
- Snadno vzniknou mírně odlišné kopie.

Co je funkce?

Funkce

Pojmenovaný blok příkazů, který lze opakovaně zavolat; může přijmout vstupy a vrátit výsledek.

- **Definice** popisuje, co se má při volání stát.
- **Volání** spustí tělo funkce.
- Název vyjadřuje účel: `circle_area`, ne `do_it`.

Definice a volání

```
def circle_area(diameter):  
    radius = diameter / 2  
    return 3.14159 * radius ** 2
```

```
area = circle_area(0.06)  
print(area)
```

- Za hlavičkou def je dvojtečka.
- Odsazení vymezuje tělo funkce.
- Závorky ve volání jsou podstatné: circle_area(0.06).

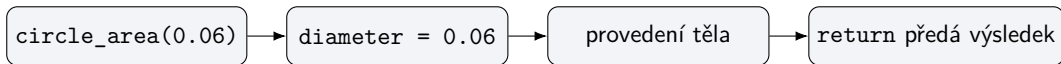
Definice sama tělo nespustí

```
def greet(name):  
    print("Hello", name)    # runs only when called  
  
print("Before")  
greet("Eva")  
print("Po")
```

Výstup

Před → Ahoj Eva → Po

Jak proběhne volání?



Každé volání získá vlastní lokální proměnné. Po návratu program pokračuje za místem volání.

Parametr není argument

Parametr jméno vstupu v definici funkce: `diameter`

Argument konkrétní hodnota předaná při volání: `0.06`

Při volání

Argument se přiřadí odpovídajícímu parametru a vznikne lokální proměnná.

Více parametrů: záleží na pořadí

```
def tube_mass(d_out, d_in, length, density):  
    area = circle_area(d_out) - circle_area(d_in)  
    return area * length * density
```

```
mass = tube_mass(0.06, 0.05, 6, 7850)
```

Poziční argumenty se přiřazují zleva doprava.

Výchozí a pojmenované argumenty

```
def tube_mass(d_out, d_in, length, density=7850):  
    area = circle_area(d_out) - circle_area(d_in)  
    return area * length * density
```

```
steel = tube_mass(0.06, 0.05, 6)  
aluminium = tube_mass(0.06, 0.05, 6,  
                        density=2700)
```

- Výchozí hodnota dovolí argument vynechat.
- Pojmenovaný argument zvyšuje čitelnost volání.

print a return nejsou totéž

```
def double_print(x):  
    print(x * 2)
```

```
def double_return(x):  
    return x * 2
```

```
a = double_print(5)    # vytiskne 10, a je None  
b = double_return(5)   # nic nevytiskne, b je 10
```

Návratová hodnota se dá skládat

- uložit: `area = circle_area(0.06),`
- rovnou použít: `mass = circle_area(d) * length * density,`
- předat jiné funkci: `print(circle_area(d)),`
- otestovat bez čtení výpisu.

Bez explicitního `return`

Python vrátí speciální hodnotu `None`.

return funkci okamžitě ukončí

```
def safe_divide(a, b):  
    if b == 0:  
        return None  
    return a / b  
    print("Sem se program nikdy nedostane")
```

- První provedený return předá výsledek volajícímu.
- Kód za ním se v dané větvi neprovede.

Více výsledků: ve skutečnosti jedna n-tice

```
def min_max(values):  
    smallest = largest = values[0]  
    for value in values[1:]:  
        if value < smallest:  
            smallest = value  
        if value > largest:  
            largest = value  
    return smallest, largest
```

```
low, high = min_max([4, 1, 9, 3])
```

Rozsah platnosti (scope)

Lokální jméno

Vzniká uvnitř funkce a zvenčí není dostupné.

Globální jméno

Je definované na úrovni modulu; funkce ho může číst.

Stejné jméno může v různých rozsazích označovat různé proměnné.

Lokální proměnná zakryje globální

`x = 10`

```
def change(x):  
    x = x + 1  
    print(x)
```

```
change(5)    # 6  
print(x)     # 10
```

Parametr `x` je při každém volání nová lokální proměnná.

Proč se vyhýbat změnám globálního stavu?

```
total = 0
```

```
def add_bad(value):  
    global total  
    total += value
```

```
def add_good(total, value):  
    return total + value
```

- Skrytá závislost ztěžuje čtení a testování.
- Výsledek čistší funkce závisí jen na předaných argumentech.

Pozor: měnitelný argument může funkce změnit

```
def add_measurement(values, value):  
    values.append(value)
```

```
data = [2, 4]  
add_measurement(data, 6)  
print(data)                # [2, 4, 6]
```

Důvod

Parametr `values` odkazuje na stejný seznam jako `data`. Pokud změnu nechceme, pracujeme s kopií.

Jak poznat dobrou funkci?

- Má jeden jasný účel a výstižné slovesné jméno.
- Vstupy dostává parametry; výsledek vrací.
- Nespoléhá zbytečně na globální proměnné.
- Je krátká natolik, že ji lze přechíst jako jeden krok algoritmu.
- Rozhraní je stabilnější než vnitřní postup výpočtu.

Funkce skládáme do většího řešení

```
def summary(values):  
    low, high = min_max(values)  
    return {  
        "mean": mean(values),  
        "median": median(values),  
        "min": low,  
        "max": high,  
    }
```

Dekompozice

Složitý problém rozdělíme na menší části s jasným rozhraním.

Vstup a výstup oddělíme od výpočtu

```
def celsius_to_fahrenheit(value):  
    return value * 9 / 5 + 32  
  
def main():  
    value = float(input("Temperature in C: "))  
    result = celsius_to_fahrenheit(value)  
    print(result)  
  
if __name__ == "__main__":  
    main()
```

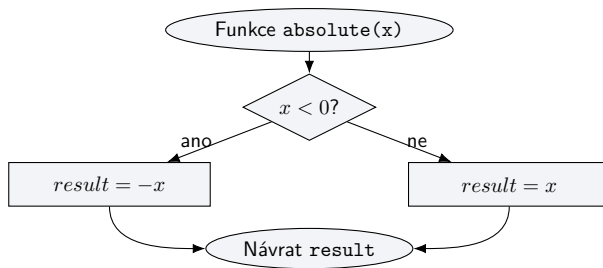
Výpočet lze znovu použít i ověřit bez klávesnice a obrazovky.

Dokumentace funkce: rozhraní před detaily

```
def tube_mass(d_out, d_in, length, density=7850):  
    """Return the mass of a hollow tube in kilograms.  
  
    Dimensions are in metres, density in kg/m^3.  
    """  
    area = circle_area(d_out) - circle_area(d_in)  
    return area * length * density
```

Docstring říká, co funkce očekává a vrací; neopisuje každý řádek.

Funkce ve vývojovém diagramu



- Začátek nese hlavičku funkce, konec její návrat.
- Diagram zobrazuje tok uvnitř jednoho volání.

Funkce lze předat jako argument

```
def apply_twice(function, value):  
    return function(function(value))
```

```
def double(x):  
    return 2 * x
```

```
print(apply_twice(double, 3))  # 12
```

Bez závorek `double` označuje samotnou funkci; `double(3)` je její volání.

Rekurze: funkce volá sama sebe

```
def factorial(n):  
    if n == 0:                # base case  
        return 1  
    return n * factorial(n - 1)
```

- Základní případ zastaví další volání.
- Rekurzivní krok převádí problém na menší instanci.
- `factorial(3)` čeká na `factorial(2)`, ta na `factorial(1)` atd.

Rekurze není automaticky rychlá

- Každé volání má vlastní parametry a lokální proměnné.
- Naivní Fibonacciho funkce opakovaně počítá stejné podproblémy.
- Pomoci může iterace nebo **memoizace**: uložení už spočítaných výsledků.

Nejdřív správnost

Vždy musí existovat základní případ a každý krok se k němu musí přibližovat.

Časté chyby

- Chybí dvojtečka nebo odsazení za def.
- Funkce je pouze pojmenovaná, ale nejsou použity závorky pro volání.
- Záměna `print` a `return`.
- Argumenty jsou v jiném pořadí než parametry.
- Použití lokální proměnné mimo funkci.
- Rekurze nemá dosažitelný základní případ.

Rychlá kontrola porozumění

- 1 Co vrátí funkce bez příkazu `return`?
- 2 Jaký je rozdíl mezi parametrem a argumentem?
- 3 Proč se globální `x` nezmění, když má funkce parametr `x`?
- 4 Proč má výpočet hodnotu vracet místo jejího vypsání?
- 5 Co musí mít každá rekurzivní funkce?

Co využijete na cvičení?

- Skládání funkcí: obsah kruhu $\rightarrow$ průřez $\rightarrow$ hmotnost trubky.
- Výchozí a pojmenované argumenty pro hustotu materiálu.
- Více návratových hodnot a funkce nad seznamem měření.
- Čisté výpočty oddělené od `input` a `print` v `main()`.
- Pro pokročilé: funkce jako argument a rekurze.

Shrnutí

- `def` vytvoří funkci; závorky ji zavolají.
- Parametry tvoří rozhraní, argumenty dodávají konkrétní hodnoty.
- `return` předá hodnotu volajícímu a ukončí funkci.
- Lokální proměnné patří jednomu volání; globální stav používejte střídmě.
- Malé funkce dovolují program číst, skládat a ověřovat po částech.